

浅谈嵌入式软件开发模式与软件架构

周治国

(浙江省开化县供电局, 浙江衢州 324300)

摘要: 嵌入式软件开发与传统的桌面型软件开发存在较大的差异, 但其平台化、分层式、构件化的设计、开发思想是完全一致的。本文主要以家庭智能终端的业务应用为例, 讲述嵌入式软件应用程序的开发模式以及在该模式下的软件模块结构。

关键词: 家庭智能终端; 仿真模拟器; 跨平台; 集成开发环境

中图分类号: TP311 **问下标识码:** A **文章编号:** 1003-9767 (2012) 11-0074-03

1. 开发模式

嵌入式软件开发与桌面型软件开发有很大的差异性, 前者需要交叉编译环境、调试需要将代码烧入目标主机等等。这给开发带来许多麻烦、且无法快速响应市场应用需求的变更。

通过平台化、套件式的交付, 降低开发成本, 提高研发效率和产品质量, 缩短产品的上市周期, 使得我们以更低的运作成本更快的响应客户需求。基于套件式的应用开发采用模拟器调试、并与开发环境完全集成将是嵌入式应用程序开发的趋势, 其应用程序开发步骤如下图 1 所示:

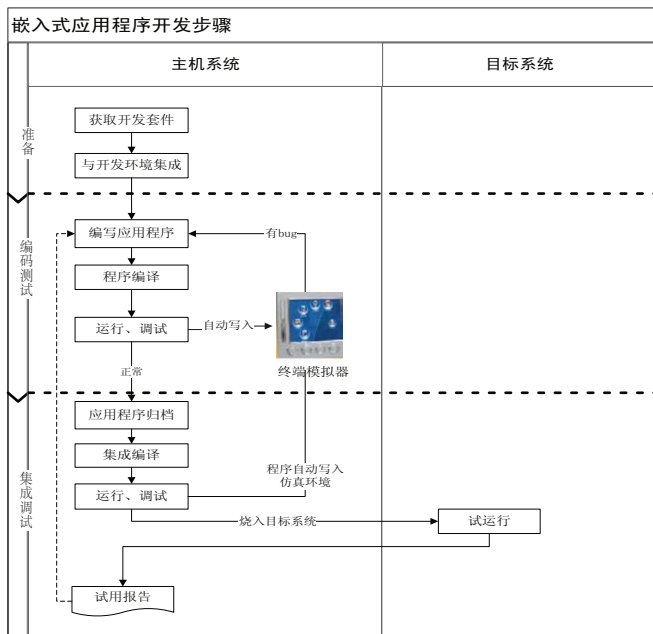


图1 嵌入式应用程序开发步骤

其中开发套件包括嵌入式操作系统、函式库（类库）、应用框架、仿真器、IDE 插件等组成。整个嵌入式开发包含两个层面的开发：开发套件的开发、应用程序开发。以上流程图为应用程序的开发步骤。在整个应用开发过程中, 只有系统集成测试后该软件在仿真模拟器运行一切良好, 才可需要烧入真实硬件环境进行试运行应用。极大的方便应用程序的开发和大大缩短开发周期。

2. 业务应用架构

以下为家庭智能终端业务应用关键需求点

- √ 系统能采集各类智能家电的电能实时数据（例如 15 分钟自动采集一次）并保存（支持 30 日以上）
- √ 用户可通过 RF 卡为各类仪表（水、电、气、热）进行预付费（充值）
- √ 用户可通过手机或遥控器方式进行对智能家电进行遥控操作
- √ 系统支持家居安防的监控
- √ 用户可通过电脑查看电能数据（本地网或互联网）

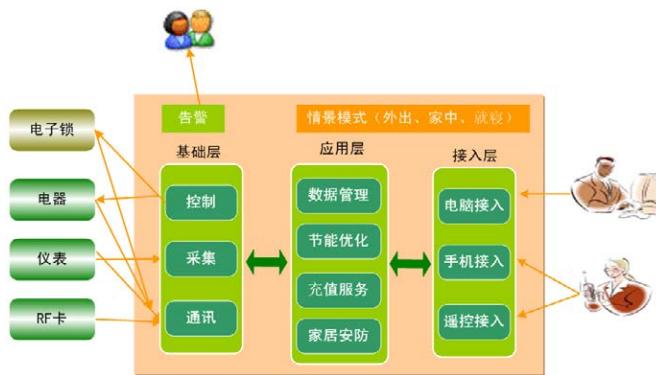


图2 家庭智能终端业务应用架构

- √ 系统能监测到设备的异常或费用不足, 会自动告警（用声音）
- √ 系统根据用户设置的情景模式, 自动启用相应的智能家居（节能）方案
- √ 系统支持一些辅助功能如播放多媒体：图片、mp3、视频、音频等文件

3. 软件体系结构

家庭智能终端嵌入式软件分应用程序层、应用框架层、函式库以及 Linux Kernel 层。其中应用程序层与应用框架层推荐采用 Java 语言实现, 因为它有着很好的跨平台特性, 同时就减少了不同的编译环境问题, 由于 Java 程序编译后并不是产生直接运行的可执行代码而是产生一个统一规格的中间码, 所以需要在每台机器上设置一个看的懂该中间码的虚拟机, 使该机器能识别 Java 的中间码而可以执行该 Java 程序。所以使用 Java 编写的程序可以在不同的机器上被执行, 这表示程序员在也不用担心所写的程序在什么机器上运行, 因此可以大大减轻程序员的负担。

LIB 层和 Linux Kernel 层主要移植现有的程序体系, 而模拟器通过模拟完整的硬件环境（如 ARM 处理器、存储、I/O 等）来虚拟化来宾平台。

3.1 应用程序

(1) 数据管理

数据管理主要负责采集、保存、查询家庭能耗数据, 其数据来源在于各类仪表如水、电、气以及智能插座、智能开关等智能测控设备。其中智能测控设备的数据来源于各类电器如冰箱、空调、彩电、照明等。数据内容分类：分步式能源数据、家电能耗数据、充值付费数据、水、气等其他能源耗用数据。

系统自动采集能耗数据, 采集频率用户可自定义（15 分钟、30 分钟、1 小时、2 小时、6 小时、24 小时）。

数据保存容量至少支持 7 日以上。

数据查询采用 HTML 文件格式的图表方式展现。支持本地（智能终端）查询和远程（通过互联网电脑）查询。

(2) 家电控制

通过该终端能实现家庭电器的控制, 如启动设备、关闭设备。

系统支持在终端上直接操作控制、也能通过遥控器、手机、计算机接入等方式进行对家电的控制操作。

(3) 家居安防

电子锁事件会自动触发终端启用安防, 自动显示视频和声音对话, 并能控制电子锁的开锁。

(4) 能源方案

实时的获取到电力公司的电价动态政策, 能源方案将根据价格政策信息以及用户的需求与意愿相结合, 指导用户合理用电。如价格上涨则家庭的储能设备如电动汽车则可向外卖电, 反之, 如价格走低, 则启用大部分家电, 并进行储能。

根据不同的情景模式, 系统会自动执行相应的家庭节能方案。

3.2 应用框架

(1) 解析

解析组件负责完成各类仪表与家电等设备的通信协议的解析与组帧。解析组件根据预先配置的规约格式文件 (XML 格式) 即可完成规约解析工作。

(2) 通讯

负责各类通讯方式如 TCP、UDP、RS232、SMS 短信等通讯组件。不同的通讯方式采用不同的组件。

(3) 任务

任务组件根据应用层配置的任务文件 (XML 格式) 的时间策略与任务内容, 定期自动执行任务。

(4) 资源管理

提供资源文件管理 (国际化多语言)、以及图形、布局等资源管理。

(5) MVC

MVC 乃模型 - 视图 - 控制框架, 其为应用程序提供介面元素如文本框 Text、列表 List、网格 Grid、按钮 Button 等等, 以及 Action 动作事件。

(6) DAO

DAO 框架提供对数据库如 SQLite 封装操作, 如连接管理、建库、建表、以及增、删、改、查等, 简化应用程序访问数据的操作。

(7) DOM4J

DOM4J 负责完成 XML 文件的解析, 应用程序与应用程序间以及应用程序与应用框架之间的交互基本都采用 XML 文件格式进行消息传递。

(8) 接入适配器

接入适配器负责不同的接入方式如 HTTP、TCP/IP、WAP 等方式的访问, 满足如通过互联网查询家电能耗数据, 通过手机方式接入控制家电等应用。

3.3 函式库

(1) 媒体库

Media 媒体库主要负责音、视频的采集、播放和媒体流的记录功能。适合开发的应用有告警提醒、安防监控以及其他多媒体应用程序。其在媒体库的内部框架简易图如图 5。

Fig.5 Media framework diagram

其包含的主要组件有内容管理 (content manager)、多路媒体引擎 (multimedia engines)、数据格式 (data format)、音频编解码器、视频编解码器、linux 接口。

(2) WebKit

对于需要浏览器的应用来说, 可选择开源的 WebKit 浏览器引擎, 它小巧、高效、稳定、易于维护是嵌入式浏览器内核的首选。

(3) 系统 C 库

系统 C 函数库, 和开发所用的交叉编译环境有关。

(4) SSL

用以保障在 Internet 上数据传输之安全, 利用数据加密 (Encryption) 技术, 可确保数据在网络之上传输过程中不会被截取及窃听, 主要用于网上付费等交易。

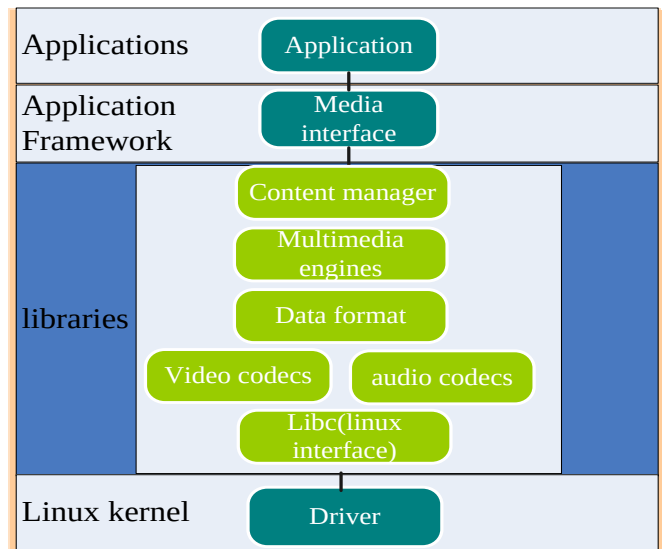


图 5 媒体库框架图

SSL 协议位于 TCP/IP 协议与各种应用层协议之间, 为数据通讯提供安全支持。SSL 协议可分为两层: SSL 记录协议 (SSL Record Protocol): 它建立在可靠的传输协议 (如 TCP) 之上, 为高层协议提供数据封装、压缩、加密等基本功能的支持。SSL 握手协议 (SSL Handshake Protocol): 它建立在 SSL 记录协议之上, 用于在实际的数据传输开始前, 通讯双方进行身份认证、协商加密算法、交换加密密钥等。

(5) 内存数据库

对于需要对大量的数据管理来说, 内存数据库是不可或缺的, 其中有多款开源数据库可选择如 FastDB、SQLite 等。

SQLite 是 C 语言编写的轻量级的、嵌入式的数据库引擎, 它是完全独立的, 不具有外部依赖性。SQLite 对 SQL92 标准的支持包括索引、限制、触发和查看。SQLite 不支持外键限制, 但支持原子的、一致的、独立和持久 (ACID) 的事务, SQLite 通过数据库级上的独占性和共享锁定来实现独立事务处理。这意味着当多个进程和线程可以在同一时间从同一数据库读取数据, 但只有一个可以写入数据。在某个进程或线程向数据库执行写入操作之前, 必须获得独占锁定。在发出独占锁定后, 其他的读或写操作将不会发生。

3.4 Linux 内核

Linux 内核主要有进程管理、内存管理、文件系统、设备控制、网络组成, Linux 的优良特性之一是在运行时扩展由内核提供的特性能力, 这意味着你可以在系统运行着的时候增加内核的功能 (也可以去除)。

(1) 系统调用接口

SCI 层提供了某些机制执行从用户空间到内核的函数调用。这个接口依赖于体系结构 (Arch), 甚至在相同的处理器家族内也是如此。SCI 实际上是一个非常有用的函数调用多路复用和多路分解服务。在 ./linux/kernel 中您可以找到 SCI 的实现, 并在 ./linux/arch 中找到依赖于体系结构的部分。

(2) 进程管理

进程管理的重点是进程的执行。一个进程就是处于执行期的程序以及它所包含的资源的总称, 在内核中, 这些进程称为线程, 代表了单独的处理器虚拟化 (线程代码、数据、堆栈和 CPU 寄存器)。在用户空间, 通常使用进程这个术语, 不过 Linux 实现并没有区分这两个概念 (进程和线程)。内核通过 SCI 提供了一个应用程序编程接口 (API) 来创建一个新进程 (fork、exec 或 Portable Operating System Interface [POSIX] 函数), 停止进程 (kill、exit), 并在它们之间进行通信和同步 (signal 或者 POSIX 机制)。

改进的 Apriori 算法在数据挖掘中的应用

王 卓

(沈阳理工大学, 辽宁沈阳 110168; 辽宁省劳动经济学校, 辽宁沈阳 110045)

摘 要: 在数据挖掘过程中, 通常来说算法选择主要考虑两方面: 一方面是因数据不同而显现的特点不同, 所以采用与其相关联的算法来进行数据挖掘; 另一方面是从实际运行系统的要求出发, 这就需要视情况而异, 选择合适的算法。数据挖掘的算法有很多种, 关联规则是数据挖掘研究的一个重要分支, Apriori 算法是关联规则挖掘中最具有影响力的算法。但因其存在着某些缺陷, 所以, 本文在分析研究关联规则挖掘中 Apriori 算法的基础上进行算法改进。改进的 Apriori 算法是一种基于划分思想的算法, 该改进算法将有效地提高数据挖掘的效率。

关键词: 数据挖掘; 关联规则; Apriori 算法; 算法改进

中图分类号: TP39 **文献标识码:** A **文章编号:** 1003-9767 (2012) 11-0076-02

1. 引言

随着信息技术的快速发展以及网络化信息时代的来临, 数据库的规模越来越大, 各种形式的数据也随之大量产生。许多重要的信息隐藏在我们看不到的地方, 那么, 如何从这些数据中发现规律并获取有用信息呢? 又该如何将这些有意义的数据用来指导我们更好的运用数据库系统呢? 这些问题备受人们关注。以往的数据库系统一般仅包含基本的录入、查询、修改功能, 而没有重视大量关联数据所存在的价值, 若想获取这些有意义的数据, 那些常用的查询、报表之类的工具已经不能满足挖掘需求。因而, 就需要一种新的数据分析处理技术来

处理海量的信息和数据, 并且从其中来抽取出有价值的潜在的知识, 为了适应信息处理新需求及社会发展多方面的迫切需求而发展起来一种新的信息分析技术, 这种技术被称为数据挖掘。它是从大量的、有噪声的、模糊的、不完全的、随机的实际应用数据当中, 将一些人们事先未知的、隐含的但又有价值的信息 & 知识提取出去的过程。在数据挖掘的众多研究领域关联规则挖掘作为一个重要分支, 是一种较为典型的数据挖掘算法。但因其存在一些固有缺陷, 主要体现在两点: 一是可能产生大量的候选集; 二是可能需要重复扫描数据库。所以, 为了满足不同的应用需求应该对传统的 Apriori 算法进行改进。

进程管理还包括处理活动进程之间共享 CPU 的需求。内核实现了一种新型的调度算法, 不管有多少个线程在竞争 CPU, 这种算法都可以在固定时间内进行操作。这种算法就称为 O(1) 调度程序, 这个名字就表示它调度多个线程所使用的时间和调度一个线程所使用的时间是相同的。O(1) 调度程序也可以支持多处理器 (称为对称多处理器或 SMP)。您可以在 `/linux/kernel` 中找到进程管理的源代码, 在 `/linux/arch` 中可以找到依赖于体系结构的源代码。

(3) 内存管理

计算机的内存是主要的资源, 处理它所用的策略对系统性能是至关重要的。内核为所有进程的每一个都在有限的可用资源上建立了一个虚拟地址空间。内核的不同部分与内存管理系统通过一套函数调用交互, 从简单的 `malloc/free` 对到更多更复杂的功能。

(4) 虚拟文件系统

虚拟文件系统 (VFS) 是 Linux 内核中非常有用的一个方面, 因为它为文件系统提供了一个通用的接口抽象。VFS 在 SCI 和内核所支持的文件系统之间提供了一个交换层,

在 VFS 上面, 是对诸如 `open`、`close`、`read` 和 `write` 之类的函数的一个通用 API 抽象。在 VFS 下面是文件系统抽象, 它定义了上层函数的实现方式。它们是给定文件系统 (超过 50 个) 的插件。文件系统的源代码可以在 `/linux/fs` 中找到。

文件系统层之下是缓冲区缓存, 它为文件系统层提供了一个通用函数集 (与具体文件系统无关)。这个缓存层通过将数据保留一段时间 (或者随即预先读取数据以便在需要是就可用) 优化了对物理设备的访问。缓冲区缓存之下是设备驱动程序, 它实现了特定物理设备的接口。

(5) 网络堆栈

网络堆栈在设计上遵循模拟协议本身的分层体系结构。回想一下, Internet Protocol (IP) 是传输协议 (通常称为传输控制协议或 TCP)

下面的核心网络层协议。TCP 上面是 socket 层, 它是通过 SCI 进行调用的。

socket 层是网络子系统的标准 API, 它为各种网络协议提供了一个用户接口。从原始帧访问到 IP 协议数据单元 (PDU), 再到 TCP 和 User Datagram Protocol (UDP), socket 层提供了一种标准化的方法来管理连接, 并在各个终点之间移动数据。内核中网络源代码可以在 `/linux/net` 中找到。

(6) 设备驱动

Linux 内核中有大量代码都在设备驱动程序中, 它们能够运转特定的硬件设备。Linux 源码树提供了一个驱动程序子目录, 这个目录又进一步划分为各种支持设备, 例如 Bluetooth、I2C、serial 等。设备驱动程序的代码可以在 `/linux/drivers` 中找到。

3.5 仿真 (模拟器)

利用现有的虚拟机技术如 QEMU、Bochs 等, 开发一款家庭智能终端仿真器, 使得该仿真器能运行在 PC 操作系统如 WIN 或 LINUX 上, 可以在 x86 平台上, 仿真 ARM 处理器、键盘、显示、USB 等主要硬件。开发人员的应用程序运行在仿真器上的效果和真实的 ARM 硬件的平台上效果完全一致。

3.6 IDE 插件

利用 IBM 的 Eclipse 的开发环境。为方便家庭智能终端的嵌入式应用开发, 专门提供一款 Eclipse 的插件以方便开发人员的代码创建、运行测试与除错功能完全集成到 Eclipse 的图形开发界面中, 该插件能自动将编写好的程序发送到仿真器中运行、测试。

参考文献:

- [1] Linux 内核设计与实现 (第 2 版) / (美) 拉芙 著; 北京: 机械工业出版社 2006.1Linux Kernel Development, Second Edition Love, R.
- [2] Android 官方网站 [http://www.android.com/Android official website](http://www.android.com/Android%20official%20website)
<http://www.android.com/>